

**Name:** Gaurika Mittal

**Batch:** 15

**Sap ID:** 590034892

## Experiment-27

### Question 1

#### Problem Statement

Write a C program to:

- Read the number of rows and columns and ensure both are positive.
- Declare a **Variable Length 2D Array (VLA)**.
- Read elements **column by column** using nested loops.
- Display the elements in **row-major order using only one loop and a linear index**.

#### Algorithm

1. Start.
2. Declare integers `rows` and `cols`.
3. Input the number of rows.
4. If `rows <= 0`, display an error message and ask again until a positive value is entered.
5. Input the number of columns.
6. If `cols <= 0`, display an error message and ask again until a positive value is entered.
7. Declare a variable-length array `a[rows][cols]`.
8. Use an outer loop for columns from 0 to `cols-1`.
9. Use an inner loop for rows from 0 to `rows-1`.
10. Read each element as `a[i][j]`.
11. To display the array, run a **single loop** from 0 to `rows*cols-1`.
12. For linear index `k`, access the element as `a[k/cols][k%cols]`.
13. Print a newline whenever `(k+1)%cols == 0`.
14. Stop.

#### Test Case Table

| Test Case | Rows     | Columns | Elements entered Column-wise | Expected Results |
|-----------|----------|---------|------------------------------|------------------|
| 1         | 2        | 3       | 123456                       | Pass             |
| 2         | 3        | 2       | 123456                       | Pass             |
| 3         | 1        | 1       | 10                           | Pass             |
| 4         | 0,then 2 | 3       | Valid Elements               | Pass             |

|   |   |           |                |      |
|---|---|-----------|----------------|------|
| 5 | 2 | -1,then 2 | Valid Elements | Pass |
|---|---|-----------|----------------|------|

## C Program

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int rows, cols, i, j, k;
```

```
    do
```

```
    {
```

```
        printf("Enter number of rows: ");
```

```
        scanf("%d", &rows);
```

```
        if(rows <= 0)
```

```
            printf("Rows must be positive.\n");
```

```
    } while(rows <= 0);
```

```
    do
```

```
    {
```

```
        printf("Enter number of columns: ");
```

```
        scanf("%d", &cols);
```

```
        if(cols <= 0)
```

```
            printf("Columns must be positive.\n");
```

```
    } while(cols <= 0);
```

```
    int a[rows][cols];
```

```
    printf("Enter array elements column by column:\n");
```

```
    for(j = 0; j < cols; j++)
```

```
    {
```

```
        for(i = 0; i < rows; i++)
```

```
        {
```

```
            printf("a[%d][%d] = ", i, j);
```

```
            scanf("%d", &a[i][j]);
```

```
        }
```

```
    }
```

```
    printf("\nArray in row-major order:\n");
```

```
    for(k = 0; k < rows * cols; k++)
```

```
    {
```

```
        printf("%d ", a[k / cols][k % cols]);
```

```
        if((k + 1) % cols == 0)
            printf("\n");
    }

    return 0;
}
```

## Compilation And Execution

```
(kali㉿kali)-[~]
└─$ vi hdoc28.c

(kali㉿kali)-[~]
└─$ gcc hdoc28.c -o hdoc28

(kali㉿kali)-[~]
└─$ ./hdoc28
Enter number of rows: 2
Enter number of columns: 3
Enter array elements column by column:
a[0][0] = 1
a[1][0] = 2
a[0][1] = 3
a[1][1] = 4
a[0][2] = 5
a[1][2] = 6

Array in row-major order:
1 3 5
2 4 6
```

```
(kali㉿kali)-[~]
└─$ gcc hdoc28.c -o hdoc28

(kali㉿kali)-[~]
└─$ ./hdoc28
Enter number of rows: 3
Enter number of columns: 2
Enter array elements column by column:
a[0][0] = 1
a[1][0] = 2
a[2][0] = 3
a[0][1] = 4
a[1][1] = 5
a[2][1] = 6

Array in row-major order:
1 4
2 5
3 6
```

```
(kali㉿kali)-[~]
└─$ gcc hdoc28.c -o hdoc28

(kali㉿kali)-[~]
└─$ ./hdoc28
Enter number of rows: 1
Enter number of columns: 1
Enter array elements column by column:
a[0][0] = 10

Array in row-major order:
10

(kali㉿kali)-[~]
└─$ gcc hdoc28.c -o hdoc28

(kali㉿kali)-[~]
└─$ ./hdoc28
Enter number of rows: 0
Rows must be positive.
Enter number of rows: 2
Enter number of columns: 3
Enter array elements column by column:
a[0][0] = 1
a[1][0] = 2
a[0][1] = 3
a[1][1] = 4
a[0][2] = 5
a[1][2] = 6
```

```
Array in row-major order:
1 3 5
2 4 6

(kali㉿kali)-[~]
└─$ gcc hdoc28.c -o hdoc28

(kali㉿kali)-[~]
└─$ ./hdoc28
Enter number of rows: 2
Enter number of columns: -1
Columns must be positive.
Enter number of columns: 2
Enter array elements column by column:
a[0][0] = 1
a[1][0] = 2
a[0][1] = 3
a[1][1] = 4

Array in row-major order:
1 3
2 4
```

## Question 2

### Problem Statement

Write a C program that:

- Reads elements into a **variable-length 2D array**.
- Copies all elements into a **1D array in row-major order**.
- Displays both the 2D array and the 1D array.

### Algorithm

1. Start.

2. Input `rows` and `cols`.
3. Validate that both are positive.
4. Declare a variable-length 2D array `a[rows][cols]`.
5. Declare a 1D array `b[rows*cols]`.
6. Using nested loops, input all elements of the 2D array.
7. Set `k = 0`.
8. Traverse the 2D array row by row.
9. Copy `a[i][j]` into `b[k]`.
10. Increment `k`.
11. Display the 2D array using nested loops.
12. Display the 1D array using a single loop.
13. Stop.

## Test Case Table

| Test Case | Rows      | Columns | Inputs               | Result               |
|-----------|-----------|---------|----------------------|----------------------|
| 1         | 2         | 3       | 1 2 3 4 5 6          | 1 2 3 4 5 6          |
| 2         | 3         | 2       | 10 20 30 40 50<br>60 | 10 20 30 40 50<br>60 |
| 3         | 1         | 4       | 5 10 15 20           | 5 10 15 20           |
| 4         | 1         | 1       | 25                   | 25                   |
| 5         | 0, then 2 | 2       | 1 2 3 4              | 1 2 3 4              |

## C Program

```
#include <stdio.h>
```

```
int main()
{
    int rows, cols, i, j, k = 0;

    do
    {
        printf("Enter number of rows: ");
        scanf("%d", &rows);

        if(rows <= 0)
            printf("Rows must be positive.\n");

    } while(rows <= 0);

    do
    {
        printf("Enter number of columns: ");
        scanf("%d", &cols);
```

```

        if(cols <= 0)
            printf("Columns must be positive.\n");

    } while(cols <= 0);

    int a[rows][cols];
    int b[rows * cols];

    printf("Enter the array elements:\n");

    for(i = 0; i < rows; i++)
    {
        for(j = 0; j < cols; j++)
        {
            printf("a[%d][%d] = ", i, j);
            scanf("%d", &a[i][j]);
        }
    }

    /* Copy into 1D array in row-major order */
    for(i = 0; i < rows; i++)
    {
        for(j = 0; j < cols; j++)
        {
            b[k] = a[i][j];
            k++;
        }
    }

    printf("\n2D Array:\n");

    for(i = 0; i < rows; i++)
    {
        for(j = 0; j < cols; j++)
        {
            printf("%d ", a[i][j]);
        }
        printf("\n");
    }

    printf("\n1D Array:\n");

    for(i = 0; i < rows * cols; i++)
    {
        printf("%d ", b[i]);
    }

    printf("\n");

```

```
    return 0;  
}
```

## Compilation And Execution

```
(kali㉿kali)-[~]  
$ vi hdoc28i.c  
  
(kali㉿kali)-[~]  
$ gcc hdoc28i.c -o hdoc28i  
  
(kali㉿kali)-[~]  
$ ./hdoc28i  
Enter number of rows: 2  
Enter number of columns: 3  
Enter the array elements:  
a[0][0] = 1  
a[0][1] = 2  
a[0][2] = 3  
a[1][0] = 4  
a[1][1] = 5  
a[1][2] = 6  
  
2D Array:  
1 2 3  
4 5 6  
  
1D Array:  
1 2 3 4 5 6
```

```
(kali㉿kali)-[~]  
$ gcc hdoc28i.c -o hdoc28i  
  
(kali㉿kali)-[~]  
$ ./hdoc28i  
Enter number of rows: 3  
Enter number of columns: 2  
Enter the array elements:  
a[0][0] = 10  
a[0][1] = 20  
a[1][0] = 30  
a[1][1] = 40  
a[2][0] = 50  
a[2][1] = 60  
  
2D Array:  
10 20  
30 40  
50 60  
  
1D Array:  
10 20 30 40 50 60
```

```
(kali㉿kali)-[~]  
$ gcc hdoc28i.c -o hdoc28i
```

```
(kali㉿kali)-[~]  
$ ./hdoc28i  
Enter number of rows: 1  
Enter number of columns: 4  
Enter the array elements:  
a[0][0] = 5  
a[0][1] = 10  
a[0][2] = 15  
a[0][3] = 20
```

```
2D Array:  
5 10 15 20
```

```
1D Array:  
5 10 15 20
```

```
(kali㉿kali)-[~]  
$ gcc hdoc28i.c -o hdoc28i
```

```
(kali㉿kali)-[~]  
$ ./hdoc28i  
Enter number of rows: 1  
Enter number of columns: 1  
Enter the array elements:  
a[0][0] = 25
```

```
2D Array:  
25
```

```
1D Array:  
25
```

```
(kali㉿kali)-[~]  
$ gcc hdoc28i.c -o hdoc28i
```

```
(kali㉿kali)-[~]  
$ ./hdoc28i  
Enter number of rows: 0  
Rows must be positive.  
Enter number of rows: 2  
Enter number of columns: 2  
Enter the array elements:  
a[0][0] = 1  
a[0][1] = 2  
a[1][0] = 3  
a[1][1] = 4
```

```
2D Array:  
1 2  
3 4
```

```
1D Array:  
1 2 3 4
```